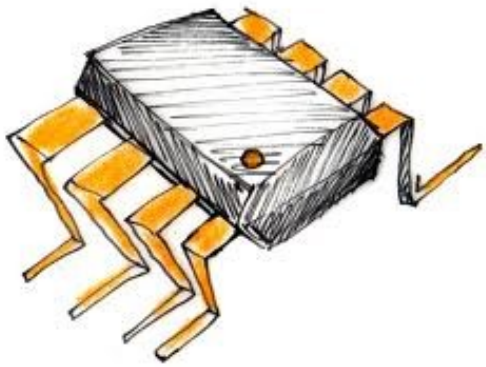
MIKROPROCESORSKI UPRAVLJAČKI UREĐAJI



Praktikum za laboratorijske vežbe

Profesor:
dr Vojin Ilić

Asistenti:
Bojan Jorgovanović
Ksenija Baraković
Anja Vranješević

Vežba 2 – Portovi

Cilj vežbe:

- Savladati rad sa portovima

Struktura programa

Program za mikrokontroler se, za razliku od programa na računaru, nikada ne završava – ne postoji operativni sistem kome bi se vratila kontrola, pa uređaj radi sve dok je uključen. Zbog toga se `main` funkcija uvek sastoji iz dva dela:

1. **Inicijalizacija** – izvršava se samo jednom, pri uključenju uređaja. Ovde se podešavaju portovi, tajmeri, prekidi i ostale periferije i postavljaju se početne vrednosti promenljivih.

2. **Beskonačna `while` petlja** – izvršava se sve dok je uređaj uključen.

Svaki prolaz kroz beskonačnu petlju sastoji se iz četiri koraka, koji se uvek izvršavaju istim redom:

1. **Očitavanje ulaza** – stanja svih ulaza (npr. tastera) se očitavaju jednom, na početku prolaza, i čuvaju u promenljivama.

2. **Logika** – na osnovu očitanih ulaza i stanja sačuvanog u memoriji određuje se novo stanje sistema i vrednosti koje treba postaviti na izlaze. U ovom koraku se ne pristupa direktno portovima, već samo promenljivama.

3. **Osvežavanje izlaza** – izračunate vrednosti se upisuju na izlaze (npr. diode).

4. **Komunikacija, dijagnostika, logovanje i sl.** – slanje i prijem podataka (npr. preko serijske komunikacije), provera ispravnosti rada, beleženje događaja i drugi pomoćni poslovi.

Jedan prolaz kroz beskonačnu petlju naziva se **ciklus skeniranja** (engl. *scan cycle*). Ovakva podela ima nekoliko prednosti. Pošto se ulazi očitavaju samo jednom po ciklusu, logika uvek radi sa istim, doslednim stanjem ulaza, čak i ako se stanje pina promeni dok se logika izvršava. Slično, izlazi se menjaju samo jednom po ciklusu, na kraju, pa na njima nema kratkotrajnih međuvrednosti dok se logika računa. Program je pregledniji, jer se tačno zna u kom delu se nalazi koji posao.

Primer:

```
void main(void)
{
    // Inicijalizacija
    // - podešavanje portova, tajmera, prekida...
    // - početne vrednosti promenljivih

    while (1)
    {
        // 1. Očitavanje ulaza
        //    stanje_tastera = očitano stanje sa porta;
```

```
        // 2. Logika
        //    na osnovu stanja tastera i stanja u memoriji
        //    izračunati novo stanje_dioda

        // 3. Osvežavanje izlaza
        //    upis stanje_dioda na port

        // 4. Komunikacija, dijagnostika, logovanje...
    }
}
```

Rad sa portovima

Mikrokontroler AT89C51RC2 ima četiri 8-bitna ulazno-izlazna porta: `P0`, `P1`, `P2` i `P3`. Da bi se portovima pristupalo iz programa, potrebno je na početku uključiti biblioteku `<at89c51rc2.h>`. U njoj se nalaze definicije registara mikrokontrolera, pa i registara koji predstavljaju portove. Upisom vrednosti u registar porta postavljaju se stanja na njegovim pinovima, a čitanjem registra dobija se trenutno stanje pinova.

Pored pristupa celom portu, moguće je pristupiti i svakom bitu porta pojedinačno. Na primer, `P2_3` se odnosi na 4. bit porta `P2` (bitovi se numerišu od 0 do 7).

Primer:

```
#include <stdint.h>
#include <stdbool.h>
#include <at89c51rc2.h>

void main(void)
{
    // Inicijalizacija
    uint8_t tasteri = 0xFF;     // nijedan taster nije pritisnut
    bool taster_2 = 1;
    uint8_t diode = 0x00;

    while (1)
    {
        // Očitavanje ulaza
        tasteri = P0;           // čitanje stanja celog porta P0
        taster_2 = P0_1;        // čitanje samo 2. bita porta P0

        // Logika
        if (taster_2 == 0)      // taster na 2. bitu je pritisnut
        {
            diode = 0x0F;       // bitovi 0-3 na 1, bitovi 4-7 na 0
        }
        else
        {
            diode = ~tasteri;   // svetle diode pritisnutih tastera
        }

        // Osvežavanje izlaza
```

```
        P2 = diode;             // upis u ceo port P2
        // P2_3 = 0;            // upis samo u 4. bit porta P2
    }
}
```

GPIO biblioteka

Umesto da se registrima portova pristupa direktno iz `main` funkcije, rad sa portovima izdvajamo u posebnu biblioteku (`gpio.h` i `gpio.c`). Na taj način glavni program ne zavisi od toga na koji port je povezan koji uređaj, već samo poziva funkcije sa jasnim nazivima. Ako se hardver promeni (npr. diode se povežu na drugi port), izmena se pravi samo na jednom mestu, u biblioteci. Kod je pregledniji, lakše se testira i može se ponovo koristiti u narednim projektima.

Biblioteka treba da:

- inicijalizuje portove (postavi početna stanja ulaza i izlaza),
- omogući upis vrednosti na izlazni port (diode na portu `P2`),
- omogući čitanje vrednosti sa ulaznog porta (tasteri na portu `P0`).

Fajl `gpio.h` sadrži deklaracije funkcija koje su dostupne ostatku programa:

```
#ifndef GPIO_H
#define GPIO_H

#include <stdint.h>

void init_gpio(void);          // inicijalizacija portova
void setP2(uint8_t port2);     // upis vrednosti u port P2
uint8_t getP0(void);           // čitanje vrednosti porta P0

#endif
```

Fajl `gpio.c` sadrži implementacije tih funkcija i jedini direktno pristupa registrima portova:

```
#include "gpio.h"
#include <at89c51rc2.h>

void init_gpio(void)
{
    // ...
}

void setP2(uint8_t port2)
{
    // ...
}

uint8_t getP0(void)
{
```

```
    // ...
}
```

U glavnom programu je dovoljno uključiti `gpio.h` i pozivati ove funkcije. Primer iz prethodnog poglavlja, napisan pomoću GPIO biblioteke:

```
#include <stdint.h>
#include "gpio.h"

void main(void)
{
    // Inicijalizacija
    uint8_t tasteri = 0xFF;     // nijedan taster nije pritisnut
    uint8_t diode = 0x00;
    init_gpio();                // inicijalizacija portova

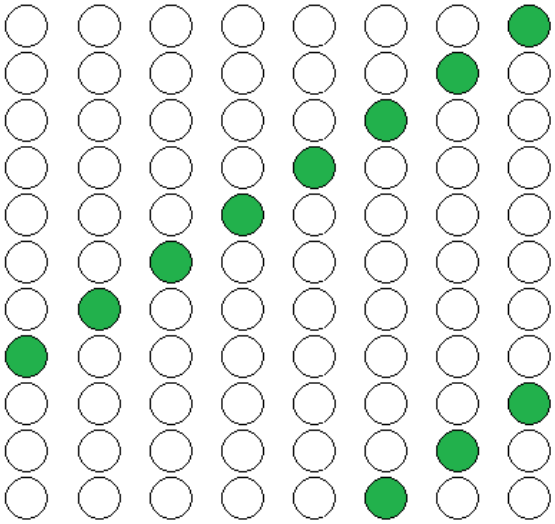
    while (1)
    {
        // Očitavanje ulaza
        tasteri = getP0();      // čitanje stanja porta P0

        // Logika
        if ((tasteri & 0x02) == 0)  // taster na 2. bitu je pritisnut
        {
            diode = 0x0F;       // bitovi 0-3 na 1, bitovi 4-7 na 0
        }
        else
        {
            diode = ~tasteri;   // svetle diode pritisnutih tastera
        }

        // Osvežavanje izlaza
        setP2(diode);           // upis u port P2
    }
}
```

Zadatak 1:

Napisati program koji će naizmenično uključivati i isključivati LED diode kao na slici 1. Period između dva stanja je 1 sekunda.



Slika 1:

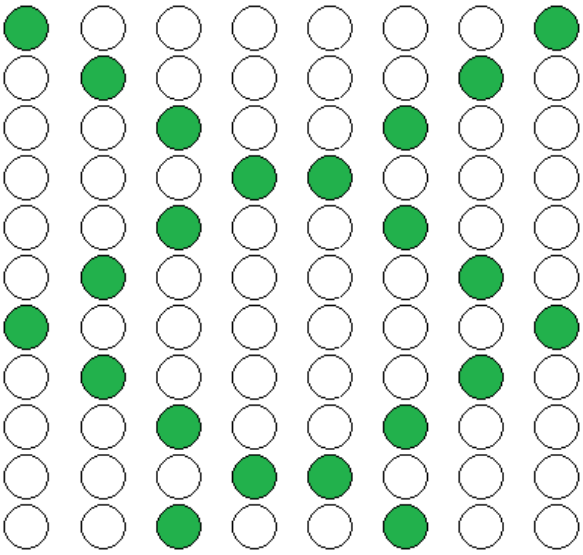
Napomena 1: Kašnjenje između dva susedna stanja možemo uraditi pomoću funkcije `delay`. Funkcija je realizovana pomoću dve ugnježdene petlje, koje će zauzeti procesorsko vreme otprilike jednu sekundu.

Funkcija `delay`:

```
void delay(void) // funkcija delay
{
    uint8_t i, j;
    for (i = 0; i < 200; i++)     // spoljašnja petlja
        for (j = 0; j < 200; j++); // unutrašnja petlja
}
```

Zadatak 2:

Napisati program koji će naizmenično uključivati i isključivati LED diode kao na slici 2. Period između dva stanja je 1 sekunda.



Slika 2:

Zadatak 3:

Proširiti program iz prethodnog zadatka tako da se njegovo izvršavanje kontroliše pomoću dva tastera: start i stop. Kada korisnik pritisne taster start, program se pokreće i diode se uključuju kao na slici 2 sve dok se ne pritisne taster stop. Nakon pritiska na taster stop, program ostaje zaustavljen sve dok korisnik ponovo ne pritisne taster start.

Zadatak 4:

Napraviti uređaj koji će uključivati diode po određenom programu. Korisnik na raspolaganju ima tastere: start, stop, program 1, program 2, program 3, program 4, brzina 1 i brzina 2. Kada korisnik pritisne taster start, diode počinju da svetle po određenom programu. Pritiskom na taster stop prekida se izvršavanje programa. Pritiskom na program 1, 2, 3 ili 4 selektuje se odgovarajući program. Pritiskom na taster brzina 1 ili 2 selektuje se odgovarajuća brzina. Program 1 uključuje i isključuje sve diode. Program 2 uključuje diode kao na slici 1, program 3 uključuje diode kao na slici 2, a program 4 dizajner sistema treba sam da osmisli.